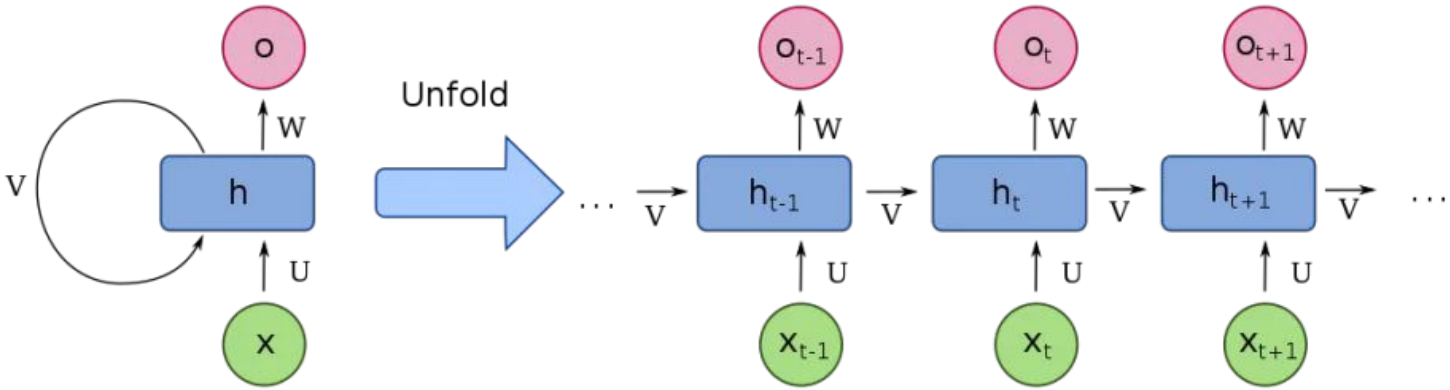
Трансформеры и LLM

Елена Кантонистова

ВШЭ

Рекуррентные сети (напоминание)

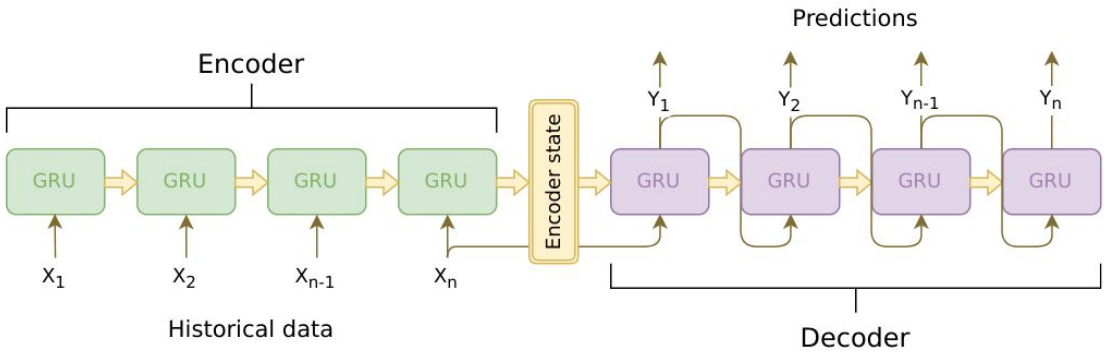
Рекуррентные нейронные сети (RNN) последовательно читают текст и агрегируют в скрытом состоянии h информацию о прочитанном.



Рекуррентные сети в задаче seq2seq

Рассмотрим задачу машинного перевода:

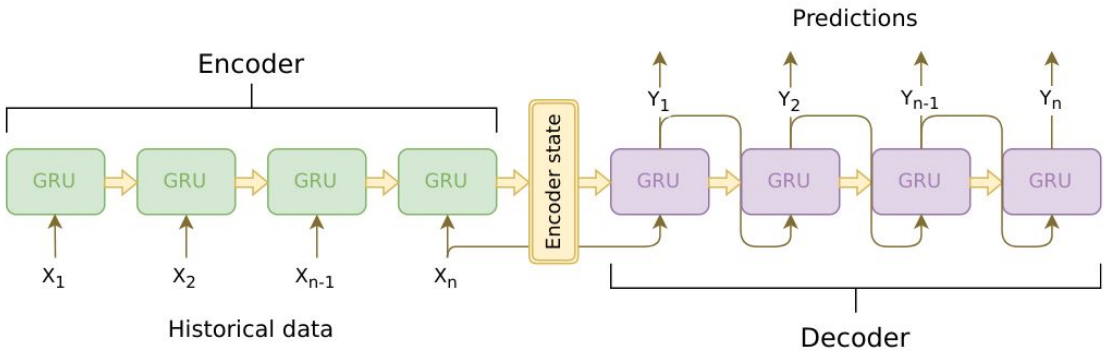
- На вход поступает текст на языке-оригинале
- На выходе - перевод



Рекуррентные сети в задаче seq2seq

В задаче seq2seq архитектура имеет структуру encoder-decoder:

- Encoder последовательно считывает текст и агрегирует информацию в вектор
- Decoder, получая на вход вектор из encoder-а и предыдущее свое состояние, последовательно генерирует целевой текст



Механизм внимания

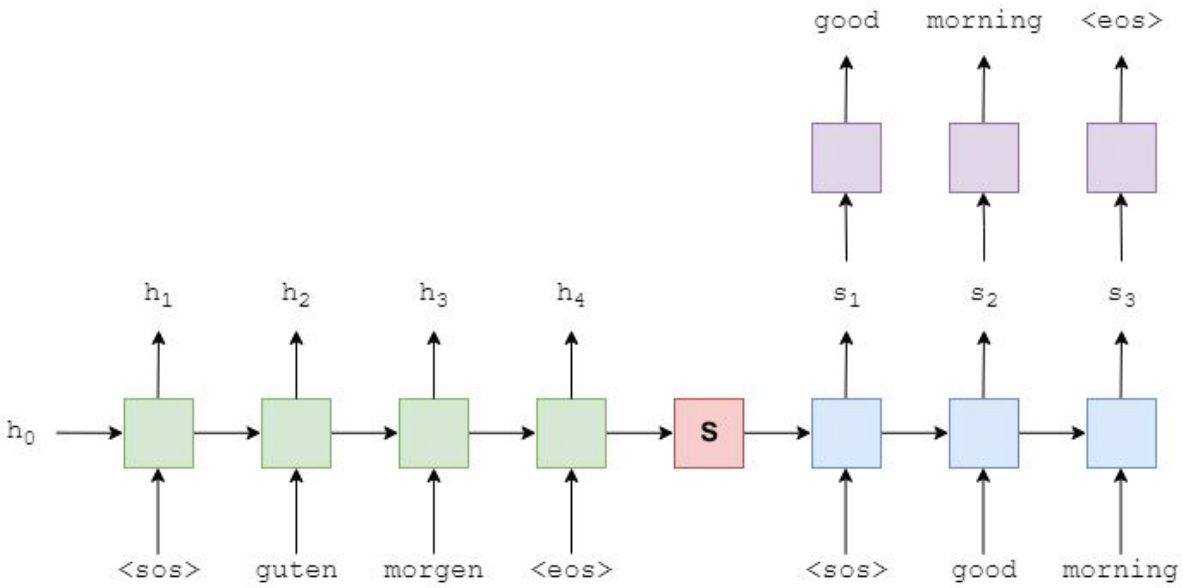
Недостатки RNN:

- Вся информация о тексте в RNN хранится в векторе скрытого состояния. Этот вектор обновляется со временем, и потому информация о словах, которые были в начале текста, быстро забывается
- Также проблемой является невозможность распараллеливания обучения RNN, то есть для получения вектора скрытого состояния модели на шаге $t+1$ нам нужно вычислить вектор скрытого состояния для шага t и так далее. Вычисления занимают много времени и неэффективно расходуют мощности GPU.

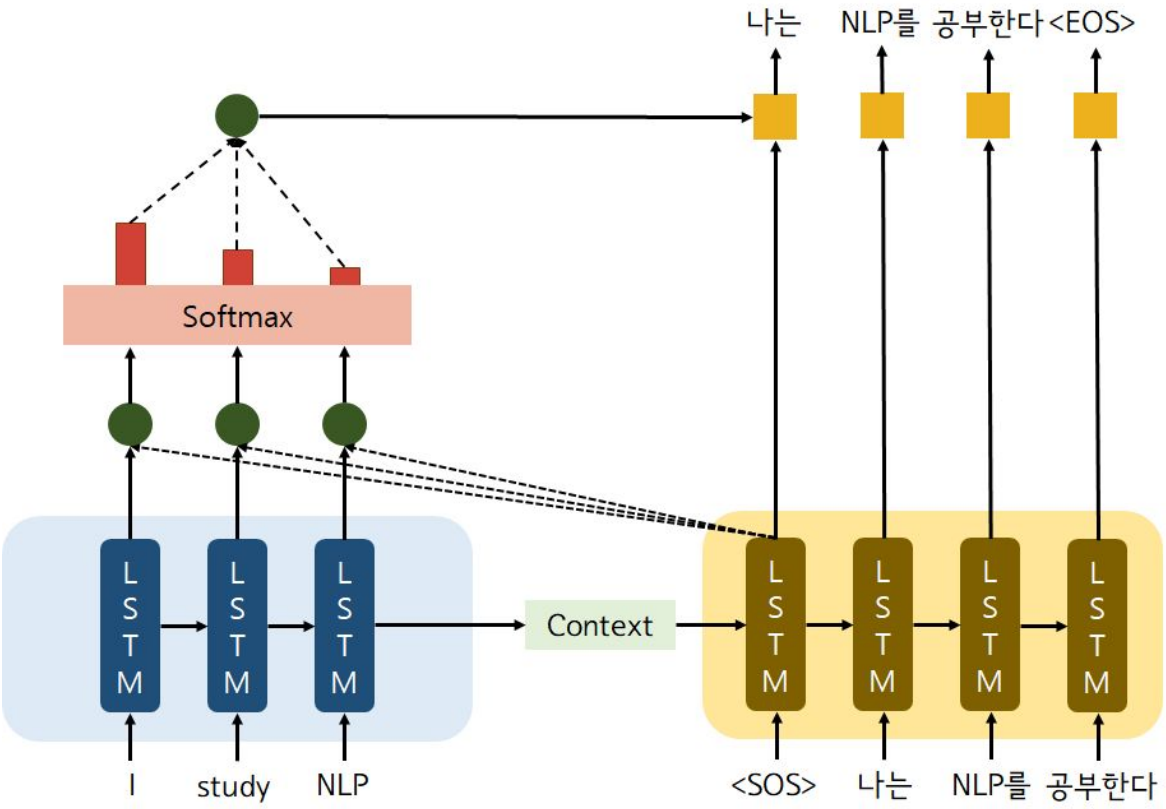
Нужен способ считывать текст таким образом, чтобы в каждый момент времени можно было получить доступ к информации о любом слове (или кусочке текста) из прошлого за фиксированное время. Также эта информация не должна теряться при дальнейшем чтении текста.

Attention

Для начала добавим внимание в декодировщик.



Attention


나는
NLP를
공부한다
<EOS>
Softmax
LSTM
LSTM
LSTM
Context
LSTM
LSTM
LSTM
LSTM
I
study
NLP
<SOS>
나는
NLP를
공부한다

Attention

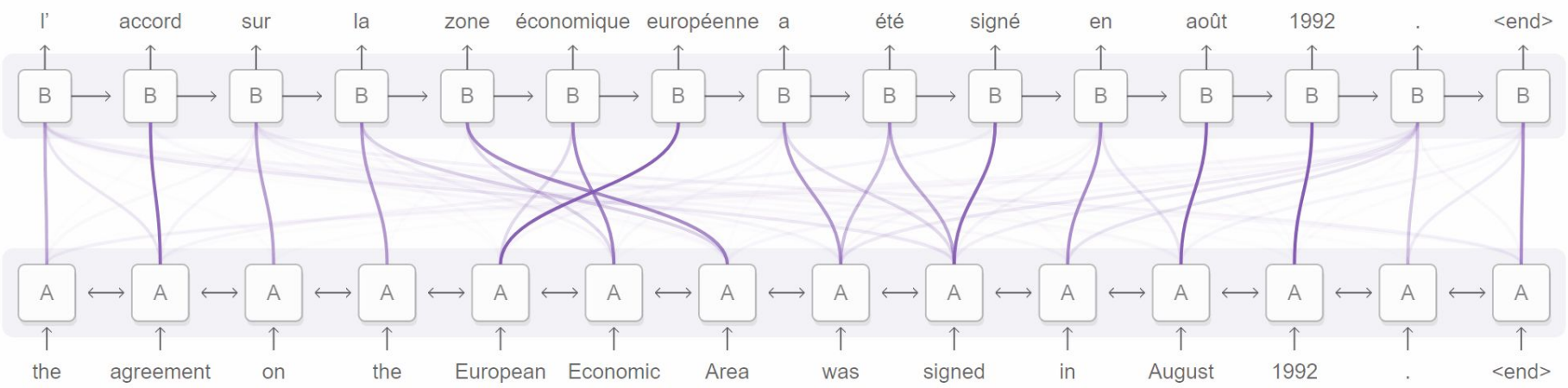


Diagram derived from Fig. 3 of Bahdanau, *et al.* 2014

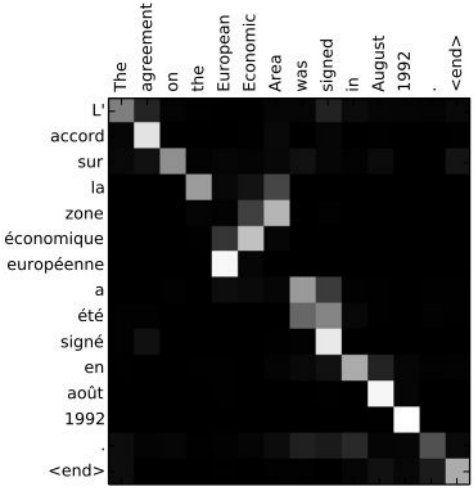
Виды attention

Существуют различные способы вычислить вектор внимания:

- **Обычное скалярное произведение** (как мы делали на предыдущем шаге): $e_i = \{(s_i, h_0), ..., (s_i, h_n)\} = [s_i h_j^T]_{j=0}^n$
- **Мультипликативное внимание с матрицей весов** W: $e_i = [s_i W h_j^T]_{j=0}^n$, где W - обучаемая матрица весов (это получается обобщение скалярного произведения)
- **Bahdanau attention (MLP)**: $e_{ij} = tanh(h_j W_1 + s_i W_2)v$, где W_1, W_2 - обучаемые матрицы весов, а v - обучаемый вектор весов.

Веса внимания несут в себе информацию о связях слов в языке-оригинала и в переводе.

Полезно визуализировать веса механизма внимания в виде тепловой матрицы.



Self-attention

Механизм внутреннего внимания (***self-attention***) используется, чтобы посмотреть на другие слова во входной последовательности во время кодирования конкретного слова. Механизм был впервые описан в статье Attention is all you need в 2017 году.

Пример.

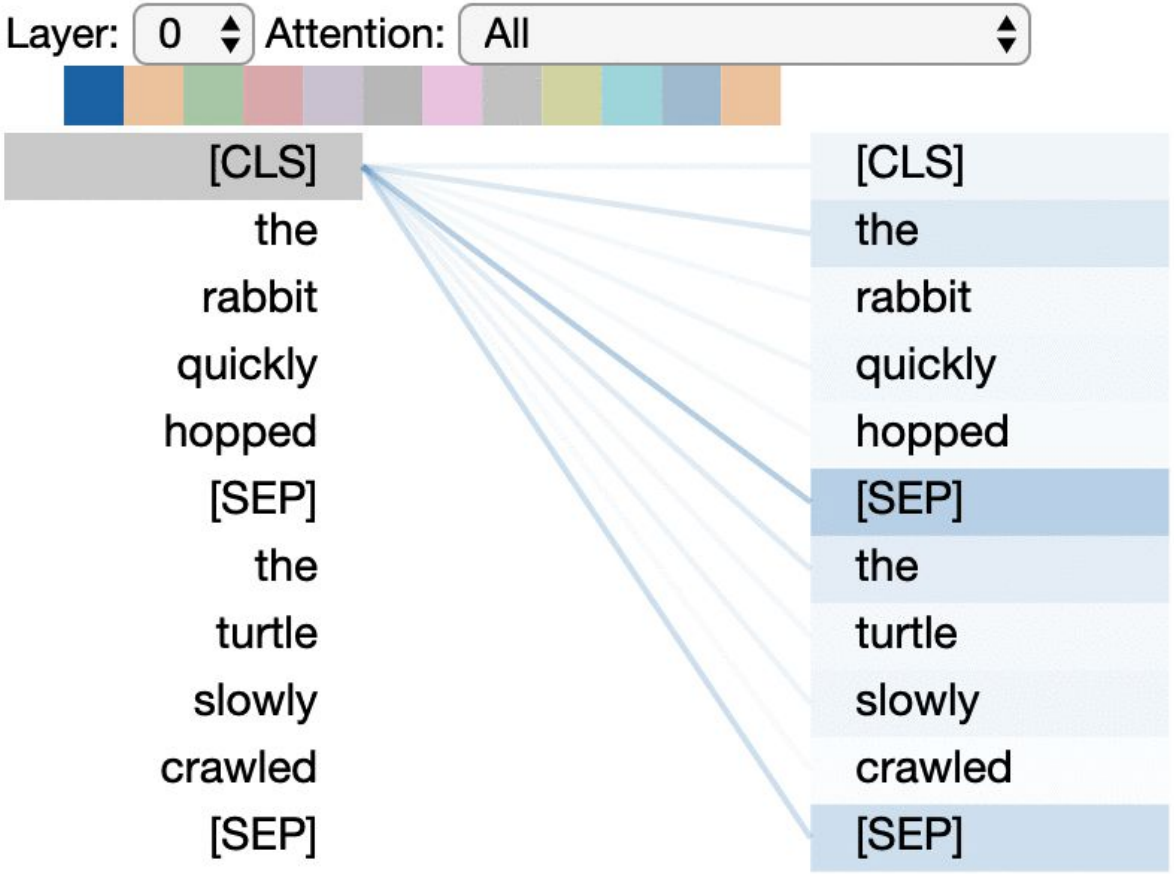
Пусть нейронная сеть получает на вход два предложения

«Мама мыла раму. Она держала в руках тряпку».

Местоимение «она» относится к маме или к раме?

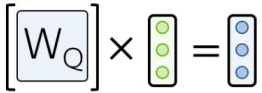
Для человека это очень простой вопрос, но для модели машинного обучения – нет. Self-attention помогает выучить взаимосвязи между токенами в последовательности, моделируя смысл и влияние других слов в тексте при обработке текущего токена.

Self-attention



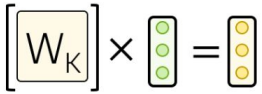
Self-attention

Each vector receives three representations ("roles")



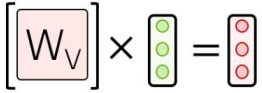
Query: vector **from** which the attention is looking

"Hey there, do you have this information?"



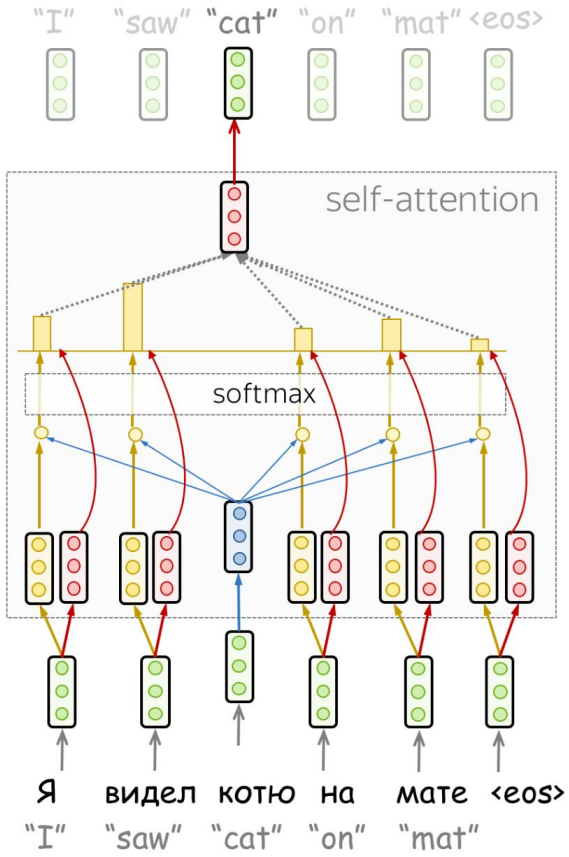
Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"



Value: their weighted sum is attention output

"Here's the information I have!"



Зачем нужны три сущности в self-attention?

Выделение этих трех сущностей нужно, чтобы разграничить эмбеддинги, задающие «направление» внимания (query, key) и смысловую часть токена (value).

- *Вектор query задает модальность «начальной точки» механизма внутреннего внимания (от какого токена направлено внимание)*
- *Вектор key – модальность «конечной точки» (к какому токену направлено внимание).*

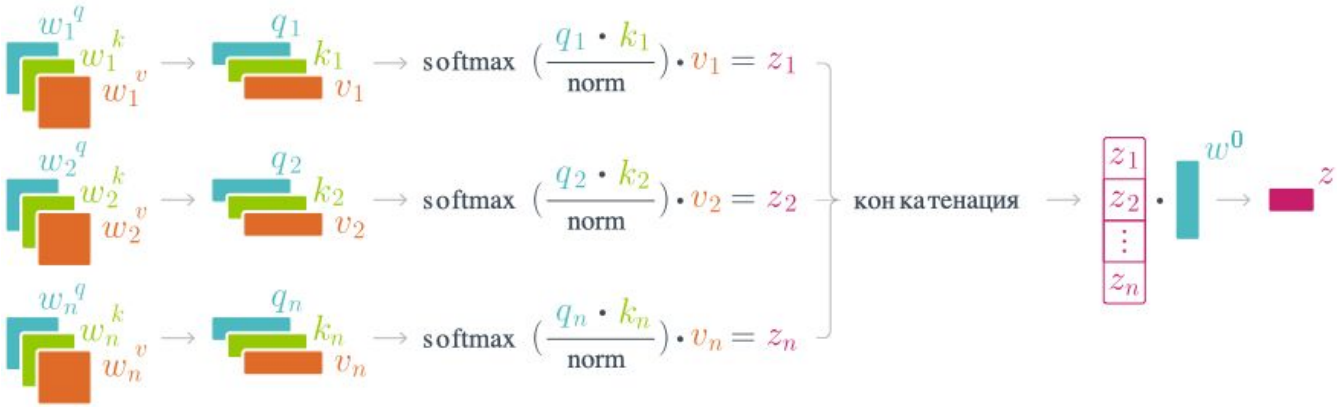
Таким образом, один и тот же токен может выступать как «начальной», так и «конечной» точкой направления внимания: self-attention вычисляется между всеми токенами в выбранном фрагменте текста.

Многоголовое внутреннее внимание

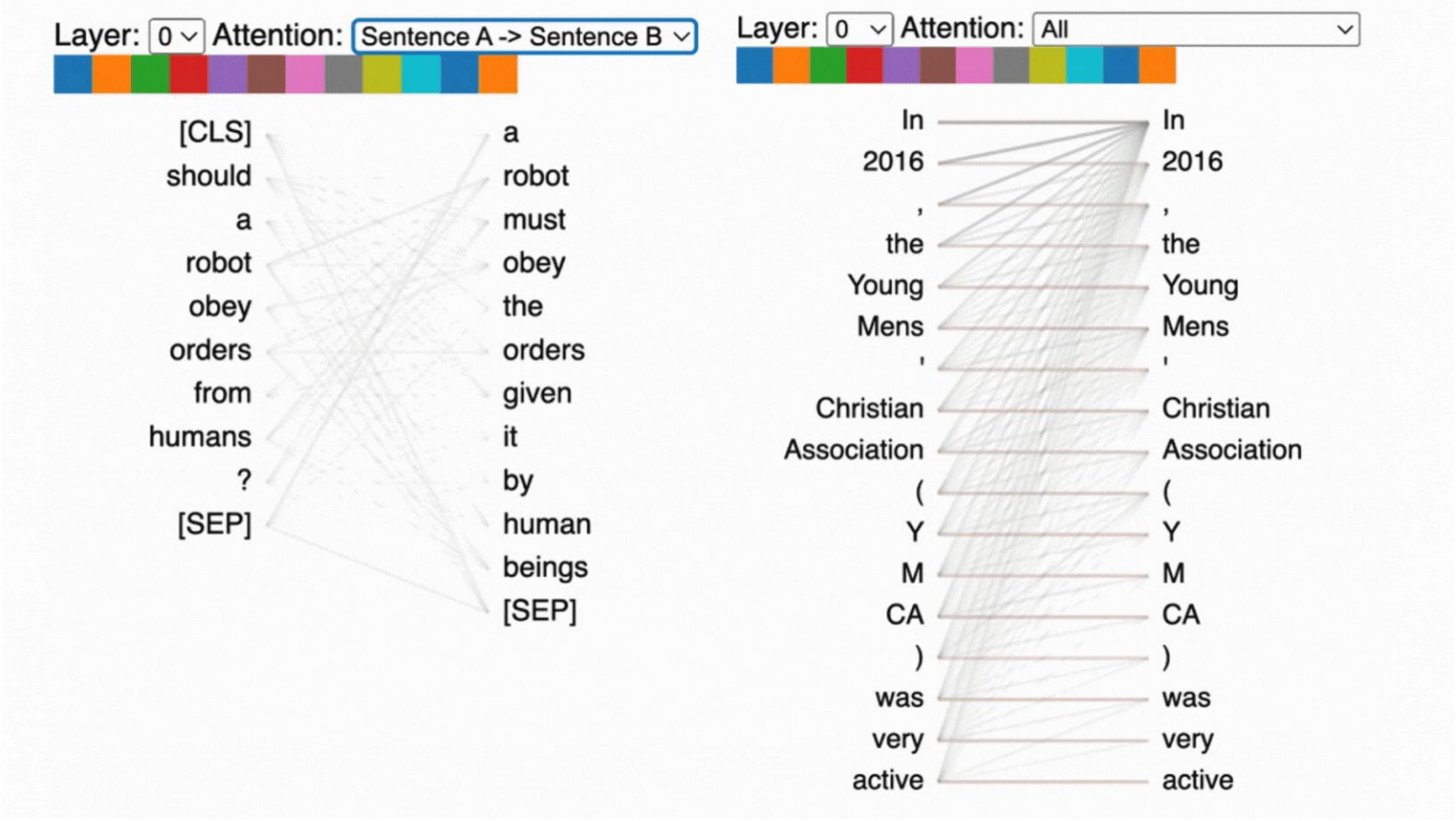
В архитектурах трансформеров обычно используют параллельно несколько self-attention блоков. Такая схема называется ***multi-head self-attention или многоголовым внутренним вниманием***.

- Вычисление self-attention происходит несколько раз с разными матрицами весов
- Затем полученные матрицы конкатенируются и умножаются на еще одну матрицу весов Wo

Это позволяет разным self-attention головам фокусироваться на разных взаимосвязях, например, одна голова может отвечать за признаковые описания, другая за действия, третья за отношения «объект-субъект».



Многоголовое внутреннее внимание



Трансформеры

Трансформеры - это архитектуры нейронных сетей, построенные с использованием механизма внимания (без рекуррентных и без сверточных слоев).

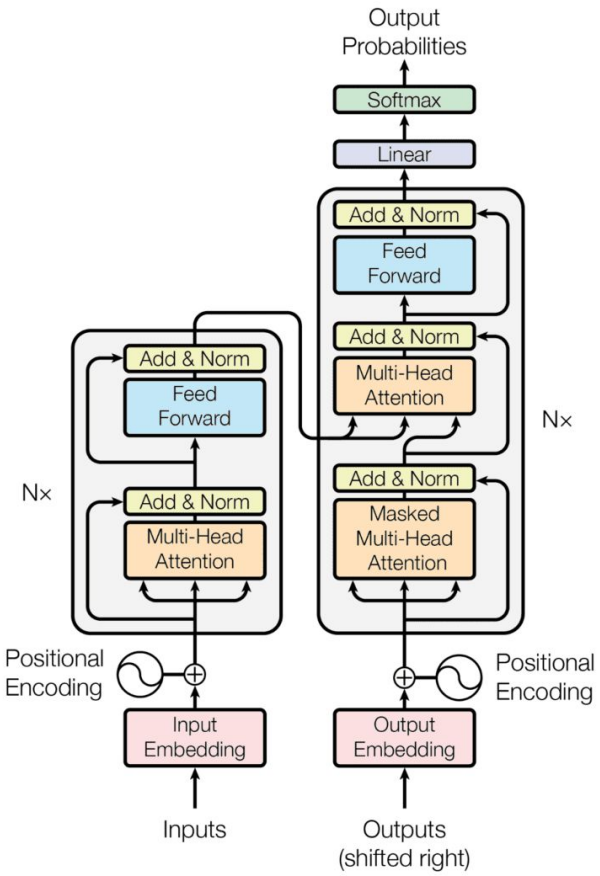
Трансформеры как архитектура появились в 2017 году и стали золотым стандартом в задачах NLP, а сейчас и в других областях (computer vision, deep learning for sound и так далее).

Зачем нам трансформеры?

- *Мы уже знаем, что рекуррентные нейронные сети содержат всю информацию о тексте в одном скрытом состоянии, что неудобно и зачастую малоинформативно для длинных текстов*
- *Кроме того, обучение RNN тяжело распараллелить в силу их идеи (последовательное чтение текста).*

Мы хотим избавиться от рекуррентности во всех частях архитектуры encoder-decoder, а также кодировать тексты с использованием механизма внимания.

Трансформеры



Верхнеуровневый процесс

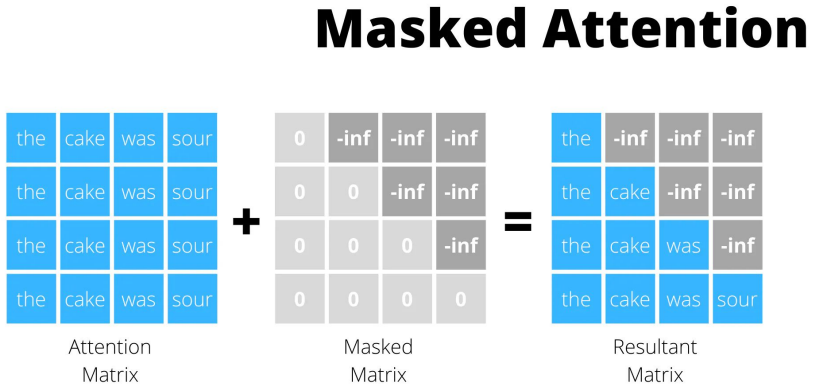
Давайте разберемся, что на ней происходит (верхнеуровнево):

- На этапе Encoder мы несколько раз используем self-attention (multi-head self-attention), чтобы получить разумные эмбеддинги, отражающие разные аспекты входного текста
- Затем на этапе Decoder на этапе генерации текущего target-токена используются эмбеддинги текста-оригинала, а также предыдущие сгенерированные декодировщиком токены (аналогично кодировщику мы получаем несколько эмбеддингов каждого target-токена, используя multi-head attention).

Decoder (декодировщик): masked attention

На этапе генерации текста декодировщиком мы не можем смотреть на весь выходной текст (так как следующие за текущим токены еще не сгенерированы), мы генерируем по одному токену за шаг, и доступ к последующим шагам на этапе обучения приведёт к утечке информации в кодировщике, а значит, к переобучению модели.

Чтобы избежать этой проблемы, при обучении к вниманию нужно применять *маску*, вручную обращая в $-\infty$ веса до применения нормализации softmax для токенов из будущего, чтобы после softmax их вероятности стали нулевыми.



*instead of words there will be attention weight

Positional Encoding

Мы избавились от рекуррентности, используя трансформеры. Но как мы теперь зафиксируем **порядок слов в тексте**? Порядок очень важен, так как именно он формирует структуру и смысл текста. Нам необходимо каким-либо образом дать сети информацию о порядке слов (токенов) в тексте. Для этого существует подход *positional encoding (позиционные эмбеддинги)*.

Positional Encoding

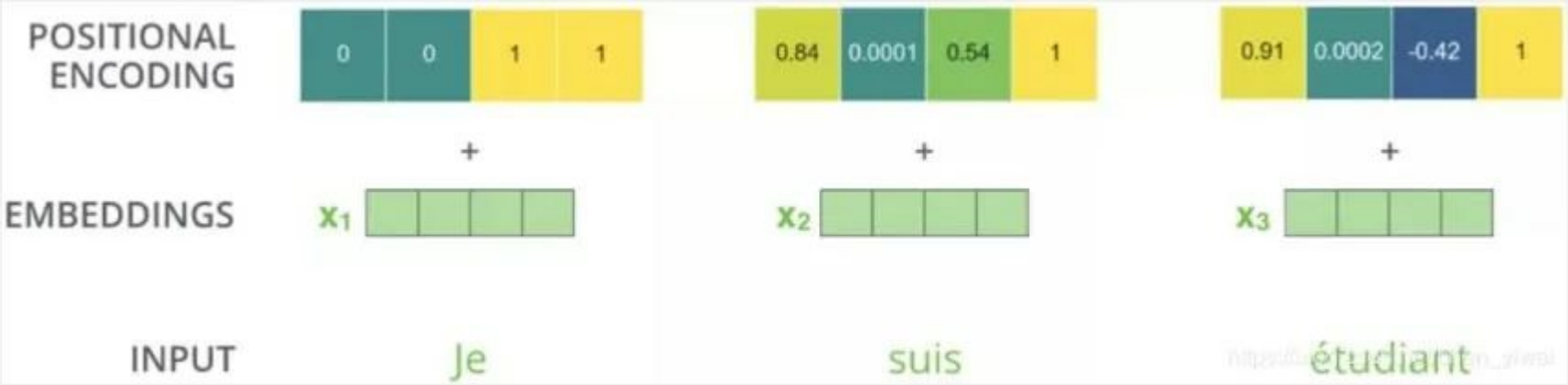
Позиционные эмбеддинги - это вспомогательные представления, которые прибавляются к обычным эмбеддингам токенов входной последовательности и позволяют слоям внимания понимать позицию токенов в тексте, а значит, различать одинаковые токены на разных местах.

Позиционные эмбеддинги могут быть:

- фиксированными, то есть, заданными по формуле
- обучаемыми - как и другие слои в нейронной сети

Практика показывает, что вполне хорошо работают и фиксированные эмбеддинги (и не надо ничего дополнительно обучать), поэтому подробнее рассмотрим их.

Positional Encoding (пример)



Positional Encoding

Фиксированные эмбеддинги задаются формулами:

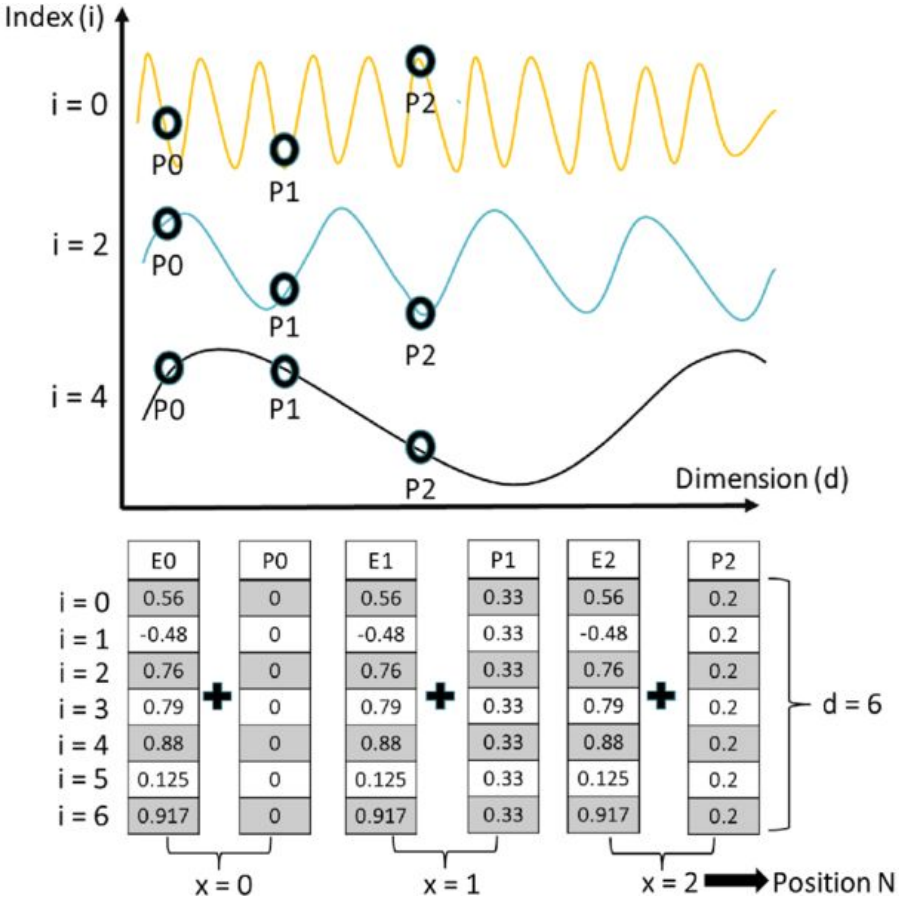
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

Здесь

- d_{model} - это размерность (dimension) модели, которая обычно соответствует *размерности эмбеддингов* токенов. Размерность модели определяет, сколько измерений будет в позиционном кодировании. Так как мы используем две компоненты (синус и косинус) для каждой позиции и каждой размерности, общее количество измерений равно размерности модели.
- i принимает значения от 0 до $d_{model}/2-1$. Мы делим размерность модели пополам, так как каждая позиционная компонента (синус и косинус) имеет свою собственную размерность.
- pos - позиция в последовательности (номер токена в предложении),
- $PE_{(pos,2i)}$ и $PE_{(pos,2i+1)}$ - компоненты позиционного кодирования для позиции pos и четной или нечетной размерности $2i$ / $2i+1$.

Positional Encoding

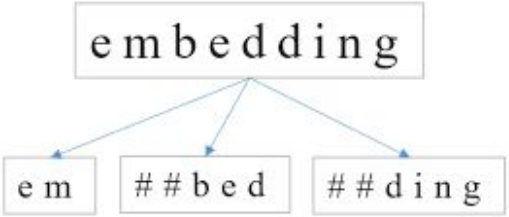


Byte Pair Encoding (BPE)

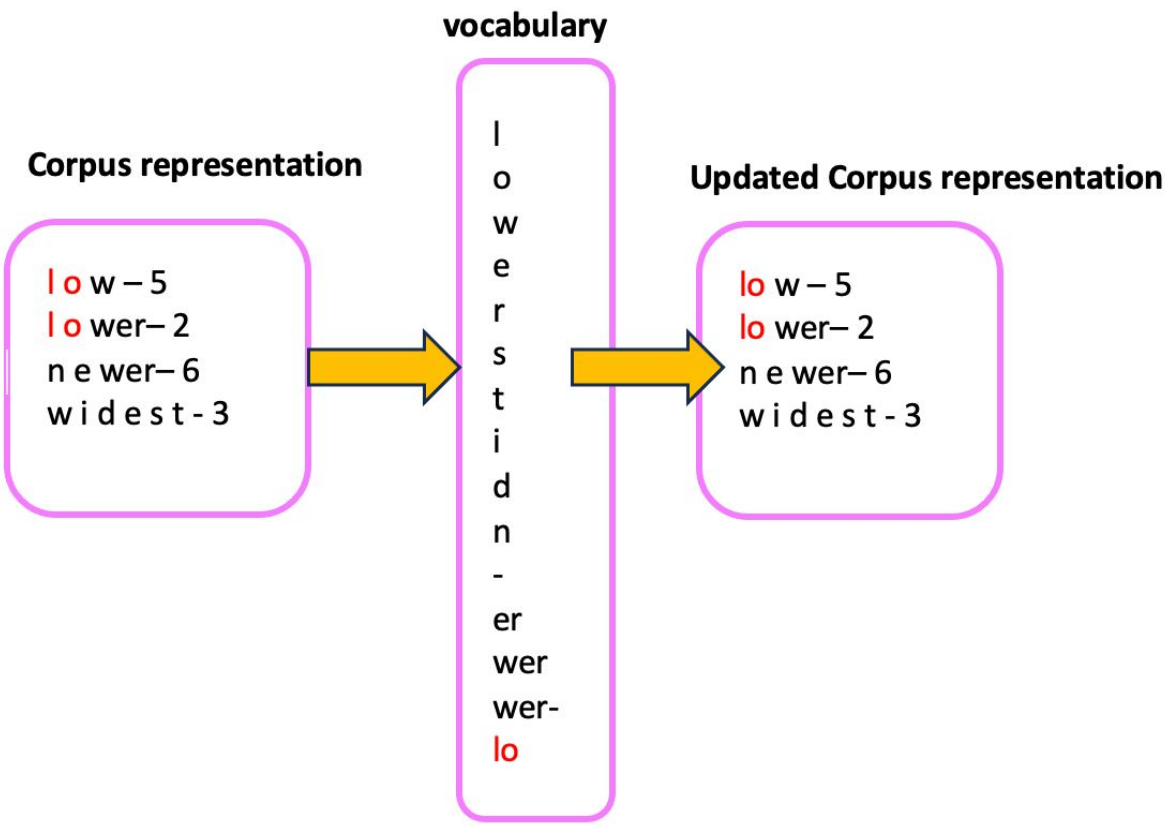
Довольно часто в текстах встречаются новые слова (которых нет в изначальном словаре, на которых модель была обучена), или же новые формы слов, а также редкие токены. Такие слова и токены невозможно закодировать - им будет присвоена метка *UNK (Unknown),* и информацию об этих токенах мы потеряем. Но обычно новое или редкое слово можно разбить на кусочки, каждый из которых уже есть в нашем словаре - будем кодировать эти кусочки, и тогда можно будет кодировать любое слово/токен, и никакая информация о тексте не потеряется. Это мотивация BPE-кодировки.

Что такое BPE?

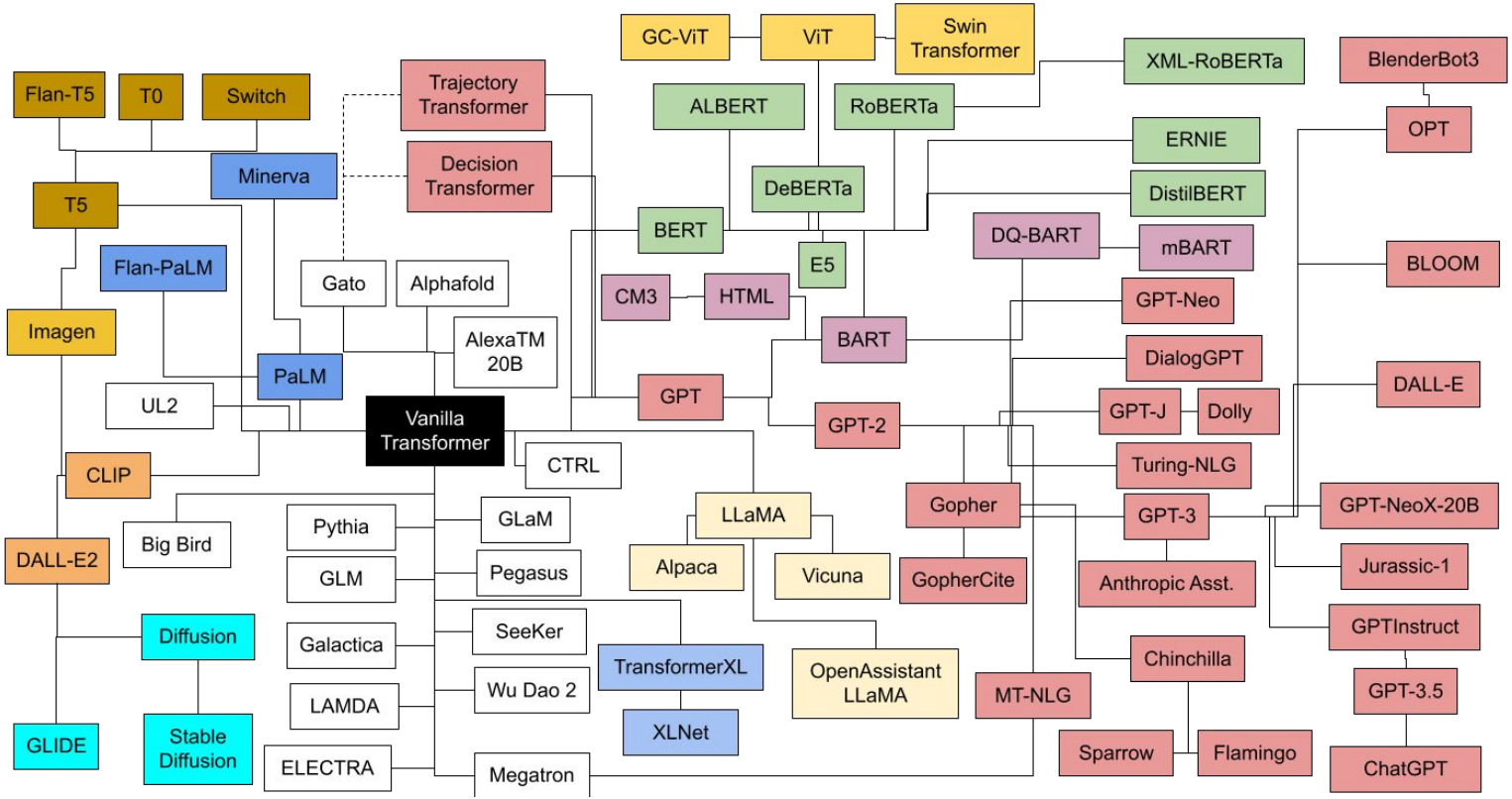
Идея состоит в том, что слова, которые часто встречаются в текстах, мы кодируем как есть (без разбиения), а слова, встречающиеся редко, будут разбиты на кусочки, и отдельные кусочки будут закодированы.



Byte Pair Encoding (BPE)

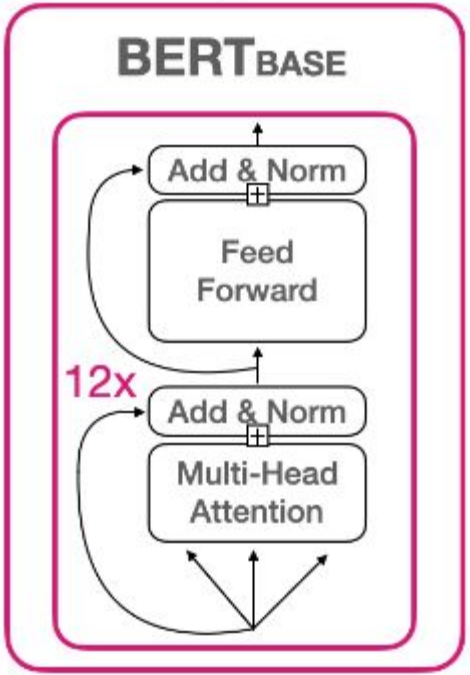


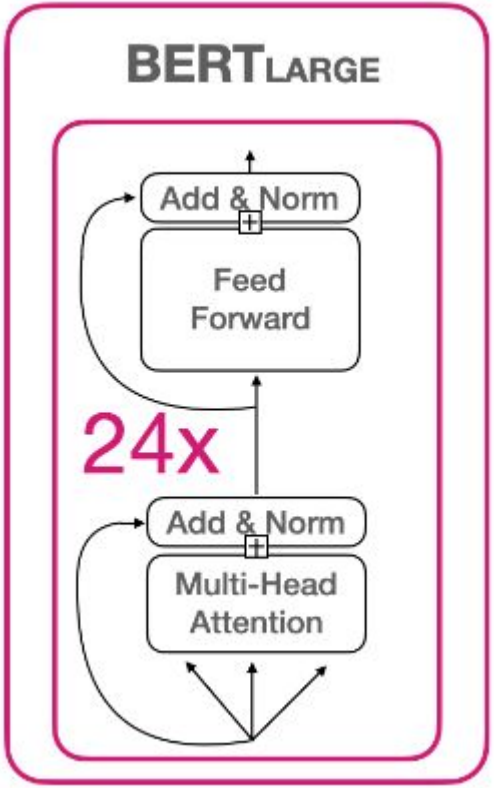
Зоопарк моделей



BERT (Bidirectional Encoder Representations from Transformers)

- BERT = энкодер от Трансформера
- base - 110M параметров
- large - 340M параметров



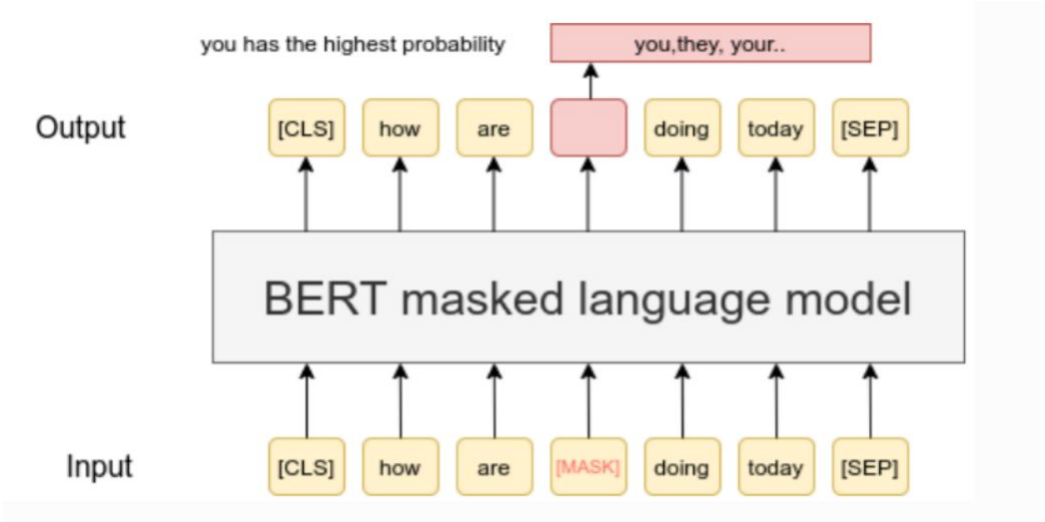


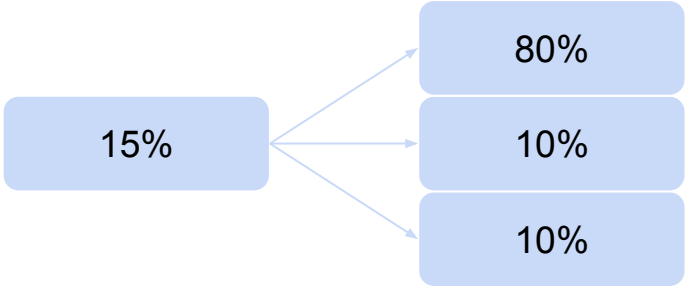
Какие задачи решает BERT?

- **Классификация текста** — например, определение тональности или темы.
- **Классификация предложений** — например, определение намерения пользователя.
- **Named Entity Recognition (NER)** — поиск сущностей в тексте (имена, организации, даты).
- **Ответы на вопросы (Question Answering)** — нахождение ответа на вопрос внутри текста.
- **Семантическое сходство текстов** — определение, похожи ли два предложения по смыслу.
  Natural Language Inference (NLI) — определение, следует ли одно предложение из другого.

Masked Language Model

Задача: Предсказать токен на месте маски [MASK]





I [MASK] machine learning

I eat machine learning

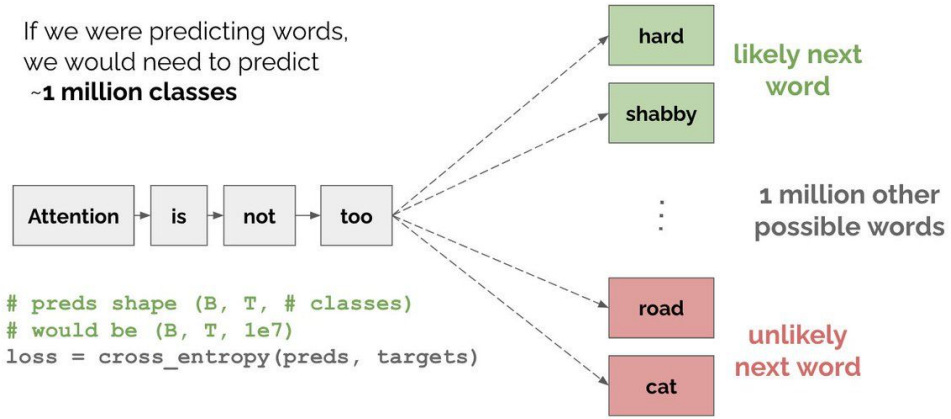
I love machine learning

BERT's performance on GLUE

Task	Average	Grammatical	Sentiment Analysis	Similarity	Paraphrase	Question Similarity	Contradiction	Answerable	Entail
BERT$_{LARGE}$	**82.1**	**60.5**	**94.9**	**86.5**	**89.3**	**72.1**	**86.7/85.9**	**92.7**	**70.1**
BERT$_{BASE}$	79.6	52.1	93.5	85.8	88.9	71.2	84.6/83.4	90.5	66.4
OpenAI GPT	75.1	45.4	91.3	80.0	82.3	70.3	82.1/81.4	87.4	56.0
Pre-OpenAI SOTA	74.0	35.0	93.2	81.0	86.0	66.1	80.6/80.1	82.3	61.7
BiLSTM+ELMo+Attn	71.0	36.0	90.4	73.3	84.9	64.8	76.4/76.1	79.8	56.8

GPT (Generative Pretrained Transformer)

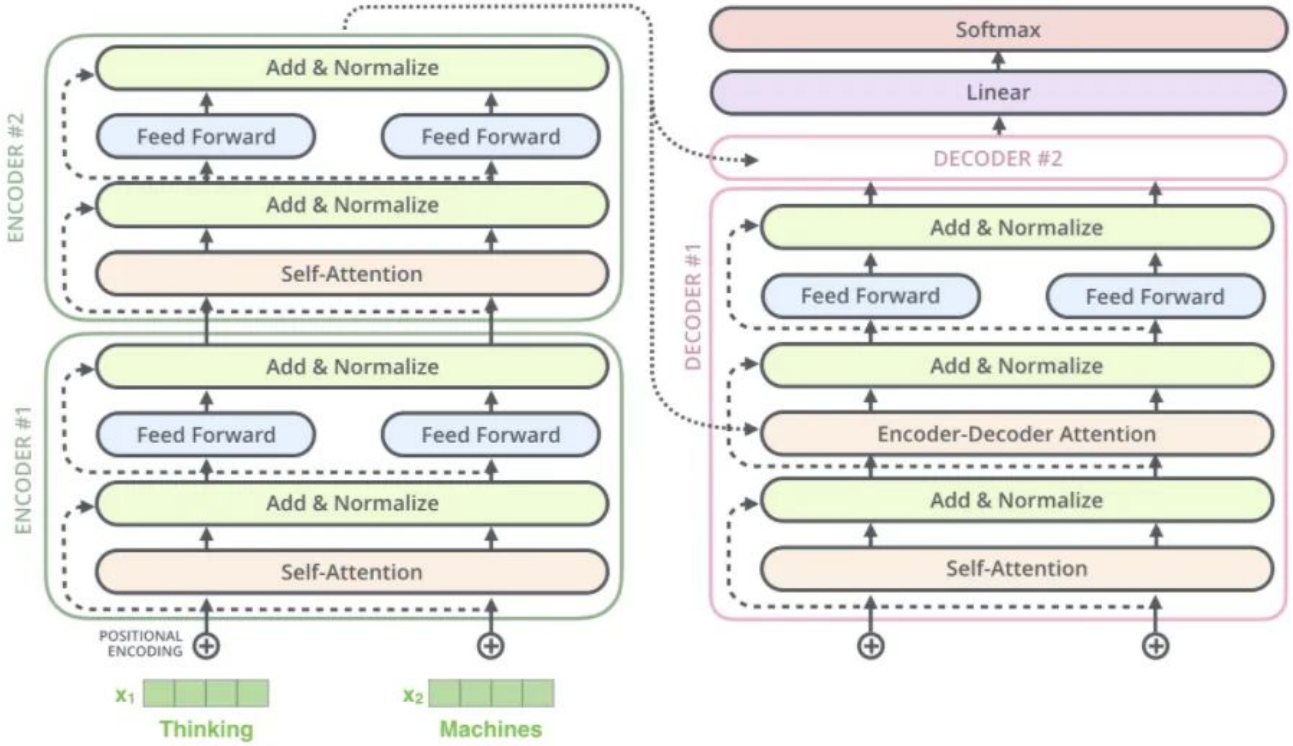
- Модель **GPT** хронологически появилась раньше, чем BERT и представляет собой обычную языковую модель, реализованную в виде последовательности слоев Transformer-декодера.
- В качестве задачи при обучении выступает обычное предсказание следующего токена (то есть многоклассовая классификация по словарю).



Какие задачи решает GPT?

- **Генерация текста** — продолжение текста или написание нового.
- **Диалоговые системы** — ответы в чат-ботах.
- **Суммаризация текста** — краткое изложение длинного текста.
- **Перевод текста** — перевод между языками.
- **Генерация кода** — написание и автодополнение программного кода.
- **Перефразирование текста** — изменение формулировки с сохранением смысла.
- **Креативные задачи** — написание историй, статей, сценариев.

T5 (Text-to-Text Transformer)

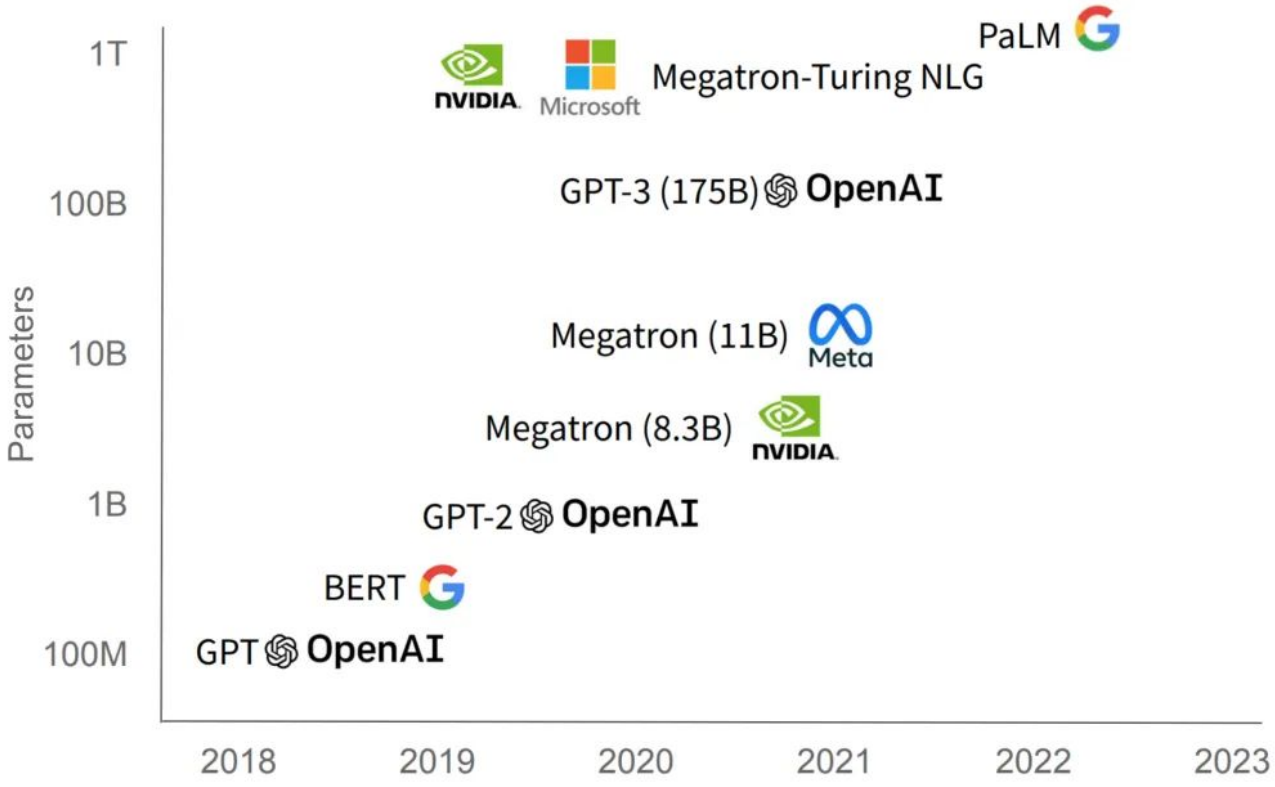


Decoder-only модели

Decoder-only модели победили в индустрии по нескольким причинам:

- **проще архитектура**
- **легче масштабировать**
- **универсальны** (через prompting)
- **хорошо работают в few-shot / zero-shot**

Количество параметров моделей



Практика-1

https://colab.research.google.com/drive/1k1AEg_6kdcKJi1BxtMTj9BVRIFIRaSm3?usp=sharing

Практика-2

https://colab.research.google.com/drive/1jSXSYD8N7uO9iXuzYNO-VMVqimT5DhD8?usp=sharing